

ARE 212 SECTION 8:

Large-sample properties of OLS

Fiona Burlig

March 10, 2016

This week, we'll take OLS for a test drive in the land of big data. Sort of. We're going to be thinking about how OLS performs in large samples - or large enough samples that they can be considered approaching asymptopia.¹ As $N \rightarrow \infty$, we can relax some of our small-sample assumptions, invoke new (weaker) assumptions, and derive cool results. Living in large-data land, otherwise known as *asymptopia*, is awesome.

Asymptopia is the best -topia

We'll demonstrate using R one of the crazier results that Max showed you in class: as long as we have linearity, population orthogonality, and the asymptotic full rank condition, our estimated $\mathbf{b}$ will both be consistent for β and be distributed Normal as $N \rightarrow \infty$. Those condition holds regardless of the distribution of the disturbances, and regardless of the distributions of left- and right-hand-side variables. We'll show off the asymptotic Normality result using four different types of off-the-wall disturbances.

Start by installing two new packages:

```
# Arrange multiple plots in a grid with gridExtra!  
install.packages('gridExtra')  
# Pareto distribution package  
install.packages('VGAM')
```

As usual, start with a setup:

```
##### GENERAL SETUP  
#nicer display of sci. notation  
options(scipen = 10)  
  
##### PACKAGES  
library(dplyr)  
library(ggplot2)  
# this new package (plus grid and gtable, which should be pre-installed,  
# allows us to make a matrix of ggplot figs. see below.  
library(grid)
```

¹Learning how to deal with Big Data, ala “data scientists” (whatever they are) is, at this point, still more useful as a real-world skill than as an academia skill. If you're interested in leaving the ivory tower once you graduate, it'd behoove you to learn some of these tools.

```

library(gtable)
library(gridExtra)

# new package to draw from a Pareto distribution
library(VGAM)

#### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

smallOLS <- function(data, y, X) {
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}

#### SET RANDOMIZATION SEED
set.seed(12345)

##### GGLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12),
  legend.key = element_blank()
)

```

As with any time we want to “prove” anything in R, we’ll turn to a simulation exercise. Side note: convincing yourself of the way econometrics works using proof-by-simulation is an extremely helpful tool in a variety of scenarios, that we don’t take advantage of enough. Simulate *all* of the things! A good Monte Carlo is a great way to silence curmudgeonly referees, too.² Let’s start by generating some data. Of course, since we’re living in asymptopia now, we’re going to generate a good sized dataset.³ As we’ve done in class before, we’ll start with a

²Not that any of them exist. We’re all bright, shiny people holding hands.

³We’ll use a population of 100,000 for now, and 1,000 samples of 10,000 observations each, since that should run pretty quickly on

“true” population, and then sample from it. Good thing `dplyr` makes this so easy! We’ll generate our X variable from a $U[0, 100]$ distribution, just to convince you that it isn’t Normality of X that’s driving our result.

```
popN <- 100000
popX <- runif(popN, 0, 100)
```

Our next step is to define our population errors (and DGP). We’ll use four types of disturbances: Normal, Uniform, Poisson, and our own crazy made-up bimodal Gamma. We’ll start by defining the bimodal Gamma distribution function.

```
# n is number of draws, prob is likelihood of binomial coming up 1
biGammaPDF <- function(n, prob) {
  # this has mean 5
  d0 <- rgamma(n, shape = 1) + 4
  # this has mean 1
  d1 <- rgamma(n, shape = 1)
  flip <- rbinom(n, size = 1, prob = prob)

  d <- d0 * flip + d1 * (1 - flip)
  # make this mean-zero for convenience
  d <- d - prob*5 - (1-prob)*1
  return(d)
}
```

Now that we’ve got that down, let’s go ahead and make each of our disturbances.

```
popError1 <- rnorm(popN)
popError2 <- runif(popN, -5, 5)
popError3 <- rpois(popN, 1) - 1
popError4 <- biGammaPDF(popN, 0.5)
```

Notice that these are each mean zero (for convenience - things go through with a non-zero mean as well):

```
c(mean(popError1), mean(popError2), mean(popError3), mean(popError4))
## [1] -0.0001166984 -0.0064061742 0.0020300000 -0.0060701929
```

Before we create our y variables, let’s take a look at what these errors look like, using our old friend `ggplot2`:

```
popErrors <- cbind(popError1, popError2, popError3, popError4) %>%
  as.tbl_df()

errorPlot1 <- ggplot(data = popErrors, aes(x = popError1)) +
  geom_histogram(color = "gray25", fill = "NA", bins = 75) + myThemeStuff +
  xlab("Normal") + ylab("Count")
```

everybody’s computers. If you’re feeling like that’s not asymptotic enough for you, go to town on your own with samples (gasp) 1 million! The largest dataset I’m currently working with has 78 million observations. Mmmm, asymptopia.

```

errorPlot2 <- ggplot(data = popErrors, aes(x = popError2)) +
  geom_histogram(color = "gray25", fill = "NA", bins = 75) + myThemeStuff +
  xlab("Uniform") + ylab("Count")

errorPlot3 <- ggplot(data = popErrors, aes(x = popError3)) +
  geom_histogram(color = "gray25", fill = "NA", bins = 75) + myThemeStuff +
  xlab("Poisson") + ylab("Count")

errorPlot4 <- ggplot(data = popErrors, aes(x = popError4)) +
  geom_histogram(color = "gray25", fill = "NA", bins = 75) + myThemeStuff +
  xlab("Binomial Gamma") + ylab("Count")

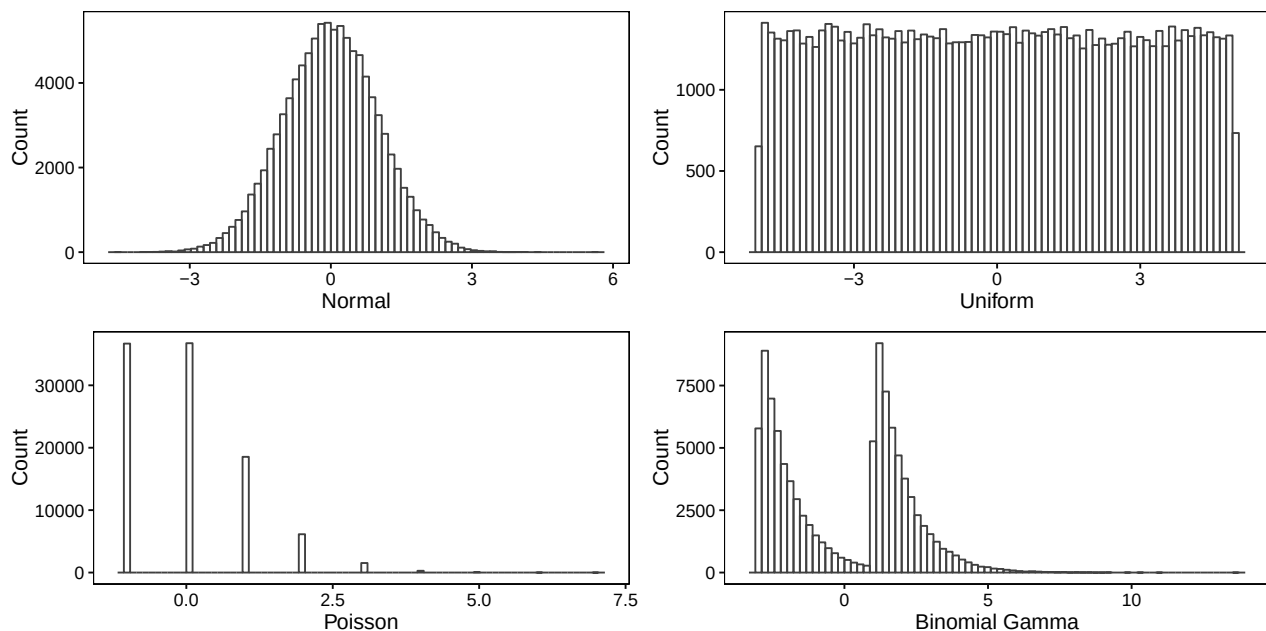
myErrors <- arrangeGrob(errorPlot1, errorPlot2, errorPlot3, errorPlot4, ncol=2)

```

```

# to print an arrangeGrob() object, use the grid.draw() fcn
grid.draw(myErrors)

```



Only one of those looks Normal! Clearly, the other three distributions violate our old small-sample Normality assumption. But not to worry - we're in asymptopia now.⁴

Now that we know what we're dealing with, we can create our population y variables.

```

popY1 <- 1 + 5*popX + popError1
popY2 <- 1 + 5*popX + popError2
popY3 <- 1 + 5*popX + popError3
popY4 <- 1 + 5*popX + popError4

```

⁴Cue *Twilight Zone* music.

We could package this in a couple of different ways. We could make a wide `tbl_df`, like this:

```
widePop <- cbind(popY1, popY2, popY3, popY4, 1, popX) %>%
  as.tbl_df()

names(widePop) <- c("popY1", "popY2", "popY3", "popY4", "ones", "popX")
```

The other thing we could do, though, is to make a *tall* `tbl_df`, with one *Y* variable, one *X* variable, a column of ones, and an indicator for which type of error created the data. We'll do this by first creating a vector indicating the type, and then telling R to treat this vector as a categorical, or **factor**, variable:

```
# creates a 400,000 vector with Normal 100k times, Uniform 100k times,
# Poisson 100k times, and Binomial Gamma 100k times
type <- c(rep("Normal", popN), rep("Uniform", popN),
          rep("Poisson", popN), rep("Binomial Gamma", popN))
#check it out
head(type)

## [1] "Normal" "Normal" "Normal" "Normal" "Normal" "Normal"

#tells R that this is categorical
type <- factor(type, levels = c("Normal", "Uniform", "Poisson", "Binomial Gamma"))

# create the tbl_df of data
popData <- cbind(c(popY1, popY2, popY3, popY4), 1, rep(popX, 4)) %>%
  as.tbl_df()
#add the type variable in
# note: must do this after, b/c otherwise we get a vector of mixed types
# and everything becomes a character
popData <- mutate(popData, type = type)

names(popData) <- c("popY", "ones", "popX", "type")
popData

## Source: local data frame [400,000 x 4]
##
##      popY  ones  popX  type
##      (dbl) (dbl)  (dbl) (fctr)
## 1  359.90467     1  72.09039 Normal
## 2  437.55116     1  87.57732 Normal
## 3  380.02502     1  76.09823 Normal
## 4  443.78582     1  88.61246 Normal
## 5  228.40648     1  45.64810 Normal
## 6   83.34465     1  16.63718 Normal
## 7  164.55752     1  32.50954 Normal
## 8  255.59483     1  50.92243 Normal
## 9  362.47958     1  72.77053 Normal
## 10 496.42817     1  98.97369 Normal
## ..      ...     ...     ...     ...
```

The nice thing about this is we can grab only rows of a certain error type with the `filter()` function, and do things to them, like this:

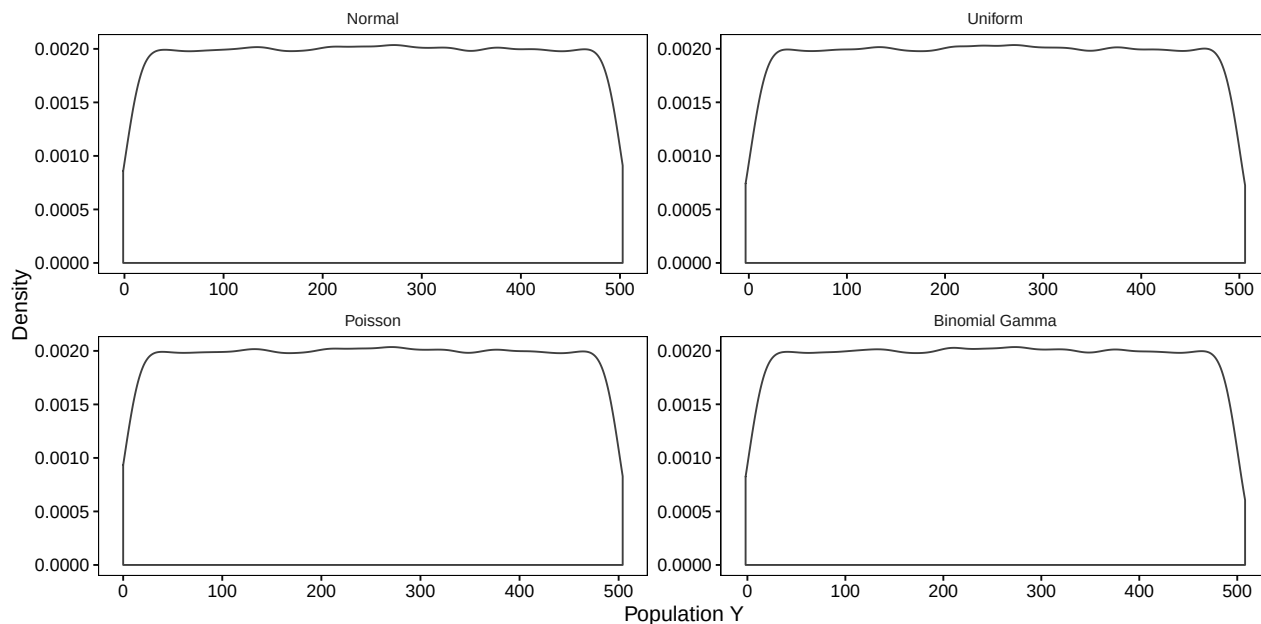
```
yMeanBiGamma <- popData %>%
  filter(type == "Normal") %>%
  summarize(mean(popY))
yMeanBiGamma

## Source: local data frame [1 x 1]
##
##   mean(popY)
##       (dbl)
## 1    251.2621
```

Now that this is set up this way, we can plot our y variables in a grid according to their type:

```
yFactorPlot <- ggplot(data = popData, aes(x = popY)) +
  geom_density(color = "gray25", fill = "NA") +
  facet_wrap(~ type, ncol = 2, scales = "free") +
  xlab("Population Y") + ylab("Density") +
  myThemeStuff +
  #turns off the gray background to the subplot headers
  theme(strip.background = element_blank())
```

yFactorPlot



That's kind of an un-informative graph, but it at least shows off our ability to do factorized plotting. Let's now turn to the main event. The first thing we need to do is to draw a sample from our population data (one for each set of errors), run OLS, and save the $\hat{\beta}$ on our X variable, and then repeat a bunch of times. We can do this really

easily using the `sample_n()` function from `dplyr` (this should look familiar to you from last week's treatment of WLS). First, let's set up the function.⁵

```
getBeta <- function(sampSi, data, y, X) {  
  # grab a sample of size sampsi from a dataset  
  sampledData <- sample_n(data, size = sampSi)  
  # grab the Y and X variables from our sample  
  sampledY <- tbl_dfGrabber(sampledData, y)  
  sampledX <- tbl_dfGrabber(sampledData, X)  
  # run OLS and save the coeff on all but the intercept  
  betahat <- smallOLS(sampledData, y, X)[-1]  
  return(betahat)  
}
```

Next, let's actually run this function using `sapply()`.⁶ The first thing we'll do, before any of these runs, is to subset our data into four parts, in each part keeping data generated using one type of errors.⁷

```
myPop1 <- filter(popData, type == "Normal")  
myPop2 <- filter(popData, type == "Uniform")  
myPop3 <- filter(popData, type == "Poisson")  
myPop4 <- filter(popData, type == "Binomial Gamma")  
  
genRegData <- function(data, y, X, nDraws, sampSi) {  
  # generate beta hats and turn them into a tbl_df  
  betahats <- sapply(1:nDraws, function(x) {  
    getBeta(sampSi, data, y, X)  
  })  
  # grab the mean & sd  
  output <- c(as.tbl_df(betahats), mean(betahats), sd(betahats))  
  names(output) <- c("betahat", "mean", "SD")  
  return(output)  
}
```

Before we go any further, let's quickly show one other result: OLS is consistent under our (fairly weak) asymptotic assumptions. We'll show this by taking 1,000 draws of sample sizes 25, 100, 1,000, and 10,000 from the population data we generated using the binomial gamma errors (just because these are the weirdest), and plotting the distribution of $\hat{\beta}$ s for each sample size. To show consistency, we need to see the distributions converging toward β as $N \rightarrow \infty$. As Dana likes to say, easy-peasy lemon-squeezy.

⁵Note the `[-1]` in the function. This is somewhat bad, because it's assuming an X matrix of first an intercept, and then other stuff. We could build in some conditionality into this function to make it less bad on this dimension, but it works well enough for this purpose, so we'll leave it. Brownie points for making it general.

⁶If you're feeling gung-ho, go ahead and parallelize this.

⁷You could imagine making your function do this for you - but I think it's clearer this way, and I'm the boss around here, at least when Max isn't around.

```

bhat25 <- genRegData(myPop4, "popY", c("ones", "popX"), 1000, 25)
bhat100 <- genRegData(myPop4, "popY", c("ones", "popX"), 1000, 100)
bhat1k <- genRegData(myPop4, "popY", c("ones", "popX"), 1000, 1000)
bhat10k <- genRegData(myPop4, "popY", c("ones", "popX"), 1000, 10000)

# combine into a dataset
consData <- cbind(bhat25$betahat, bhat100$betahat, bhat1k$betahat, bhat10k$betahat) %>%
  as.tbl_df()
names(consData) <- c("small", "medium", "large", "huge")

```

And now let's make a pretty graph. If you're not yet totally comfortable with the factor version of things, use:

```

# plot
# basic plot
consPlot <- ggplot(data = consData) +
  # draw the distributions
  geom_line(aes(x = small, linetype = "Small"),
    size = 0.5, alpha = 0.75, stat = "density", color = "deepskyblue2") +
  geom_line(aes(x = medium, linetype = "Medium"),
    size = 0.5, alpha = 0.75, stat = "density", color = "deepskyblue2") +
  geom_line(aes(x = large, linetype = "Large"),
    size = 0.5, alpha = 0.75, stat = "density", color = "deepskyblue2") +
  geom_line(aes(x = huge, linetype = "Huge"),
    size = 0.5, alpha = 0.75, stat = "density", color = "deepskyblue2") +
  myThemeStuff +
  # tell ggplot which linetype to use for which series
  scale_linetype_manual(values = c("Small" = "dotted", "Medium" = "dotdash",
    "Large" = "dashed", "Huge" = "solid")) +
  theme(legend.title = element_blank()) +
  xlab("Betahat") + ylab("Density") + ggtitle("Consistency")

```

That's an okay way of making that plot. But it's not very R-savvy. The experienced R user will tell you to make your life easier down the road use a factor variable expression of the plot instead. Let's do that:

```

consDataFac <- c(bhat25$betahat, bhat100$betahat, bhat1k$betahat, bhat10k$betahat) %>%
  as.tbl_df()

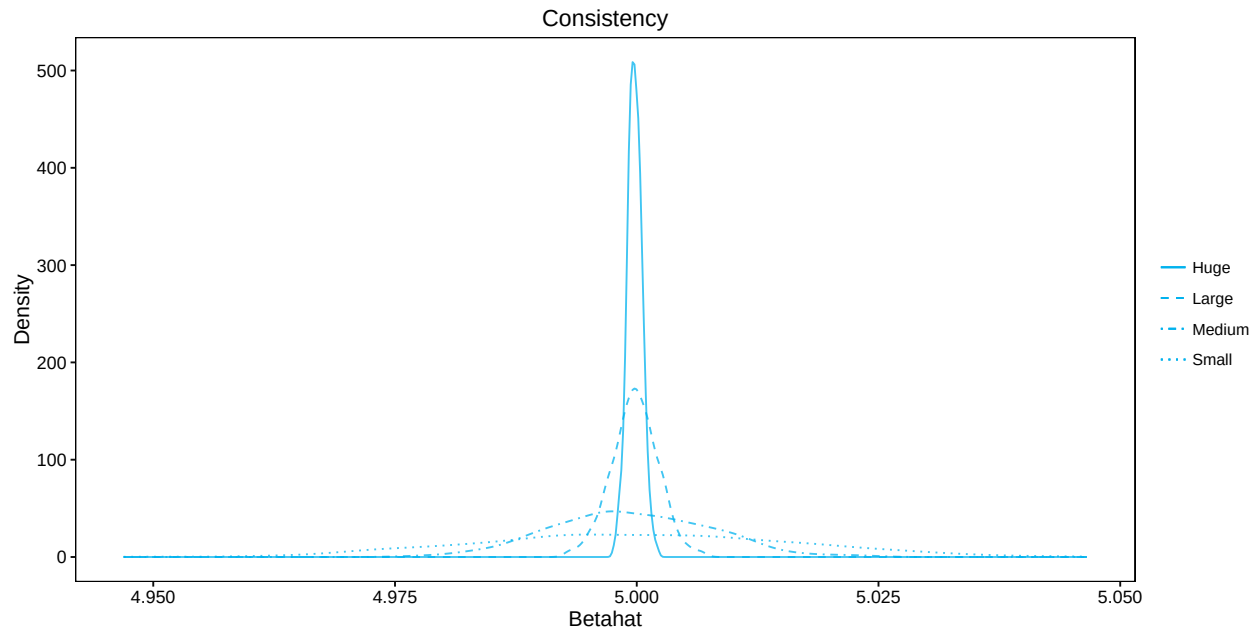
sampleSizeFac <- c(rep("Small", 1000), rep("Medium", 1000),
  rep("Large", 1000), rep("Huge", 1000)) %>%
  as.factor()

consDataFac <- mutate(consDataFac, size = sampleSizeFac)

## now the plot
consPlotFac <- ggplot(data = consDataFac, aes(x = data, linetype = size)) +
  # draw the distributions
  geom_line(size = 0.5, alpha = 0.75, stat = "density", color = "deepskyblue2") +

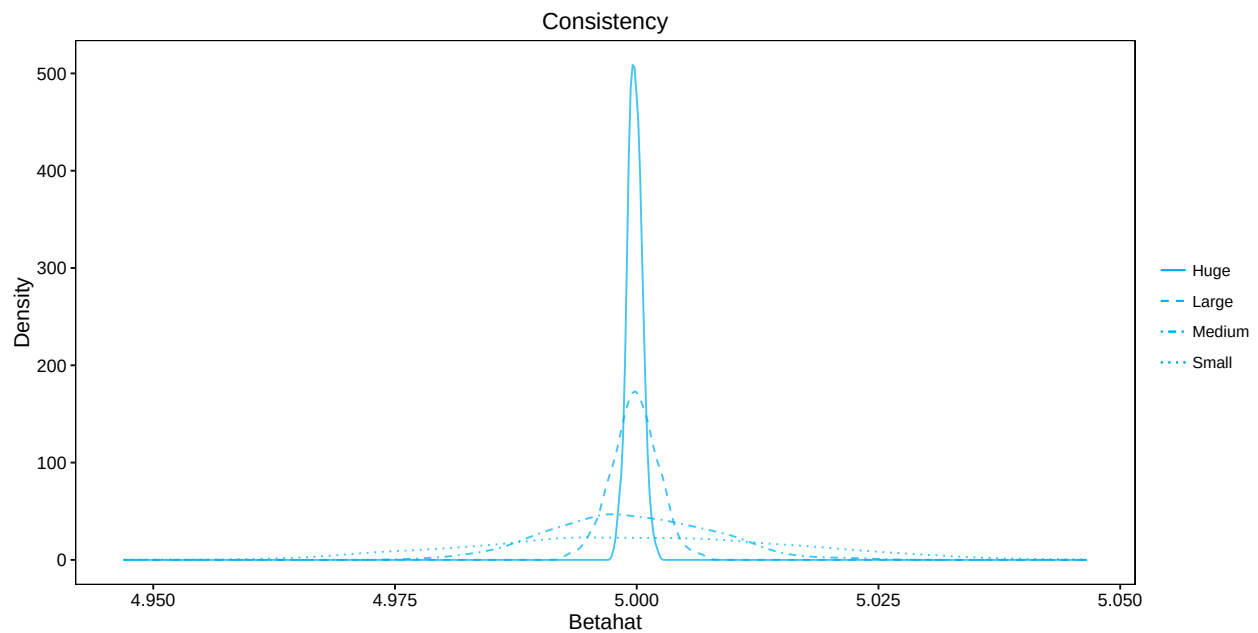
```


consPlot



```
myThemeStuff +  
  scale_linetype_manual(values = c("solid", "dashed", "dotdash", "dotted")) +  
  theme(legend.title = element_blank()) +  
  xlab("Betahat") + ylab("Density") + ggtitle("Consistency")
```

consPlot



That's way nicer code-wise. Either way, the plot is pretty nice looking.

Ahh, beautiful. Now let's get back to the more surprising part - demonstrating that $\hat{\beta}$ is asymptotically distributed Normal.

We're going to do 1,000 draws of 10,000 observations each. We're next going to take the mean and standard deviation of each $\hat{\beta}$ series, so that we can compare our generated series against a Normal distribution with the same parameters.

```
betaHats1 <- genRegData(myPop1, "popY", c("ones", "popX"), 1000, 10000)
betaHats2 <- genRegData(myPop2, "popY", c("ones", "popX"), 1000, 10000)
betaHats3 <- genRegData(myPop3, "popY", c("ones", "popX"), 1000, 10000)
betaHats4 <- genRegData(myPop4, "popY", c("ones", "popX"), 1000, 10000)
```

This is cool - we now have a bunch of point estimates from different models, as well as their mean and SD. Let's write a function to make a plot for each error type:

```
betaHatPlot <- function(betadata) {
  data <- as.tbl_df(betadata$betahat)
  names(data) <- "betahat"
  # pull out the mean & SD objects
  mymean <- betadata$mean
  mysd <- betadata$SD
  # make the plot
  # set up the base data and aesthetics
  plot <- ggplot(data = data, aes(x = betahat)) +
    # plot the histogram. density says use density rather than count
    geom_histogram(aes(y = ..density..), fill = "gray",
                   color = "lightgray", alpha = 0.8, bins = 50) +
    # make the density plot of the above histogram
    geom_line(color = "black", size = 0.5, linetype = "dashed", stat = "density") +

    # could also do that with this code, but then you get a line @ bottom:
    # geom_density(color = "black", linetype = "dashed") +

    # draw the normal distribution
    stat_function(geom = "line", fun = dnorm,
                 arg = list(mean = mymean, sd = mysd),
                 size = 0.5, color = "deepskyblue2") +
    # remove the axis labels (we'll do that jointly later)
    xlab("") + ylab("") +
    myThemeStuff

  return(plot)
}
```

And we'll create four of these plots:

```

betaHatPlot1 <- betaHatPlot(betaHats1) + ggtitle("Normal")
betaHatPlot2 <- betaHatPlot(betaHats2) + ggtitle("Uniform")
betaHatPlot3 <- betaHatPlot(betaHats3) + ggtitle("Poisson")
betaHatPlot4 <- betaHatPlot(betaHats4) + ggtitle("Binomial Gamma")

```

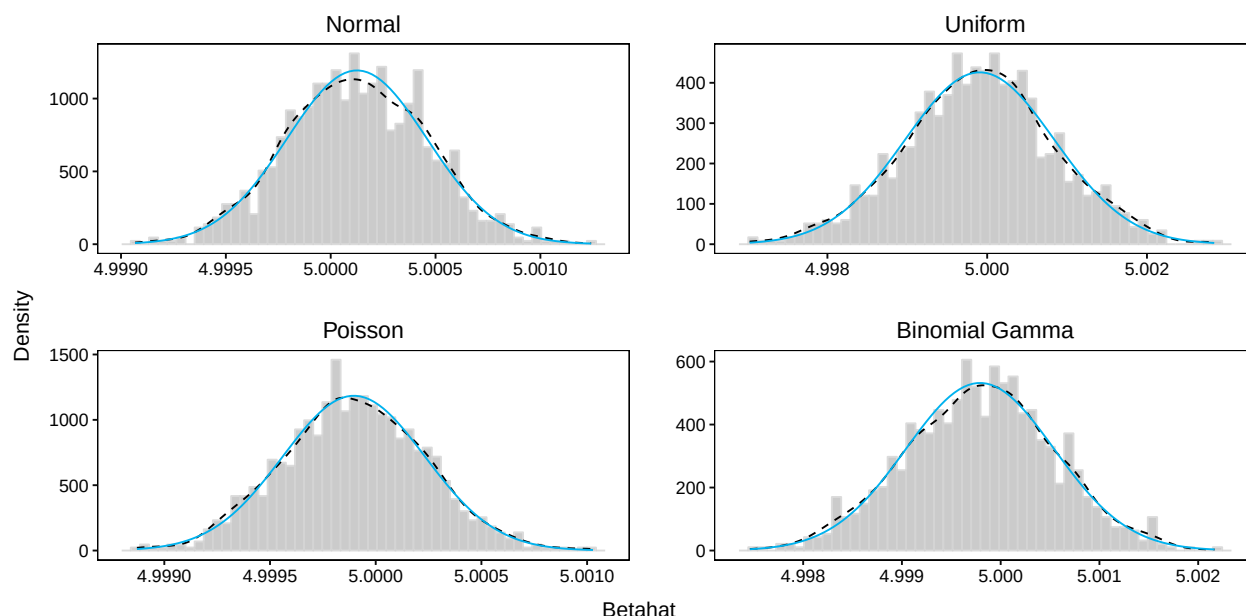
And finally, we'll put them together using the `arrangeGrob()` function from earlier.⁸

```

allBetaHats <- arrangeGrob(betaHatPlot1, betaHatPlot2,
                           betaHatPlot3, betaHatPlot4,
                           ncol=2,
                           # y axis title
                           left = textGrob("Density", vjust = 2, rot = 90),
                           # x axis title
                           bottom = textGrob("Betahat", vjust = -1))

```

```
grid.draw(allBetaHats)
```



Looks nice! Notice that regardless of the choice of error structure, things are pretty darn Normally distributed. This blew my mind the first time I saw it. Asymptopia is amazing.

“But wait!” you should be saying, “The eyeball test is great, and all, but it’s not exactly rigorous...” Busted. You’d be right. There a bunch of formal tests for different properties of distributions.⁹ Today, we’re interested in testing whether our $\hat{\beta}$ coefficients are Normally distributed, so we want a test of the null that our data came from a Normal distribution. Luckily for us, some statisticians who are smarter than me have come up with just such a

⁸You could imagine doing this with facets, like we did above, but it’s a pain to use with `stat_function()`, so I prefer `arrangeGrob()`. I generally think that `arrangeGrob()` is easier, since you can put whatever you want in each individual plot - but your code will be more efficient and concise in situations when you can use factors.

⁹One of the most commonly-used is the [Kolmogorov-Smirnov test](#), which checks for equality between two distributions.

test: the Shapiro-Wilk test.¹⁰ This test is a little bit complex to implement, and we’re a little short on time (and Max isn’t watching), so we’ll use a canned routine to calculate it.

We’ll start by going through a quick example. We’ll first randomly generate some Normal data and some Poisson data, and make sure that the Shapiro-Wilk test can correctly reject the null hypothesis when faced with a non-Normal dataset.

```
# make fake data
dataNormal <- rnorm(1000)
dataPoisson <- rpois(1000, lambda = 5)

# evaluate the S-W test, and grab the p-value
(swNormal <- (shapiro.test(dataNormal))$p.value)

## [1] 0.1065305

(swPoisson <- (shapiro.test(dataPoisson))$p.value)

## [1] 0.00000000000006287473
```

Good! The S-W test clearly failed to reject the null that our Normal data are in fact Normal, but clearly rejects the null for our Poisson data. Now that we know it works, let’s test our “actual” data.¹¹

```
# do the test
#Normal
(sw1 <- (shapiro.test(betaHats1$betahat))$p.value)

## [1] 0.9439885

#Uniform
(sw2 <- (shapiro.test(betaHats2$betahat))$p.value)

## [1] 0.8034711

#Poisson
(sw3 <- (shapiro.test(betaHats3$betahat))$p.value)

## [1] 0.9638161

#Binomial Gamma
(sw4 <- (shapiro.test(betaHats4$betahat))$p.value)

## [1] 0.8858919
```

Beautiful. We fail to reject the null of Normality in any of these cases. Looks like asymptopia is working for us.

¹⁰Want to know more? [Click here](#). #econometricsclickbait.

¹¹One thing to note: the `shapiro.test()` function is trying to be a little bit clever. It won’t accept sample sizes over 5,000, because as $N \rightarrow \infty$, the likelihood of rejecting the null increases, even if things are actually distributed Normal. See [this thread](#) for more. If your sample is larger than 5,000 observations, you could just randomly draw <5,000 observations from your data. No problem.

Second Welfare Theorem of Econometrics: Proof

Scared you, didn't I? Just kidding. Rather than talking about Pareto efficiency, we'll quickly get our hands dirty with another of Vilfredo Pareto's discoveries: the Pareto distribution.¹² In case you haven't met this distribution yet, it has a finite mean and an *infinite* variance. Yeah, you read that right. You should be afraid, because I just wrote "infinite variance."¹³

You should also be afraid because you are a great ARE 212 student and *of course* remember that our favorite Lindberg Levy Central Limit Theorem, which we used to prove that $\hat{\beta}$ will be asymptotically normal under the right conditions, requires that each element of $\mathbf{x}_i'\varepsilon_i$ has finite variance.¹⁴

Let's create some data using this scary distribution (make sure you've loaded the VGAM package).¹⁵

```
popError5Prelim <- rpareto(popN, scale = 2, shape = 1)
#demean
popError5 <- (popError5Prelim - mean(popError5Prelim)) %>%
  as.tbl_df()

# check out the summary stats
summary(popError5)

##      data
##  Min.   : -20.25
## 1st Qu.: -19.58
##  Median: -18.23
##   Mean  :   0.00
## 3rd Qu.: -14.12
##   Max.  :104327.10
```

Holy bananas! Check out that maximum value! What do these errors look like?

```
errorPlot5 <- ggplot(data = popError5, aes(x = data)) +
  geom_histogram(color = "gray25", fill = "NA", bins = 75) + myThemeStuff +
  xlab("Pareto") + ylab("Count") #+
  # we're going to cut this plot off before we get all the way out to infinity
  # xlim(-20, 5000)
```

They look scary, that's what. Can OLS save us now?

Let's create a y value and set things up for regressions.

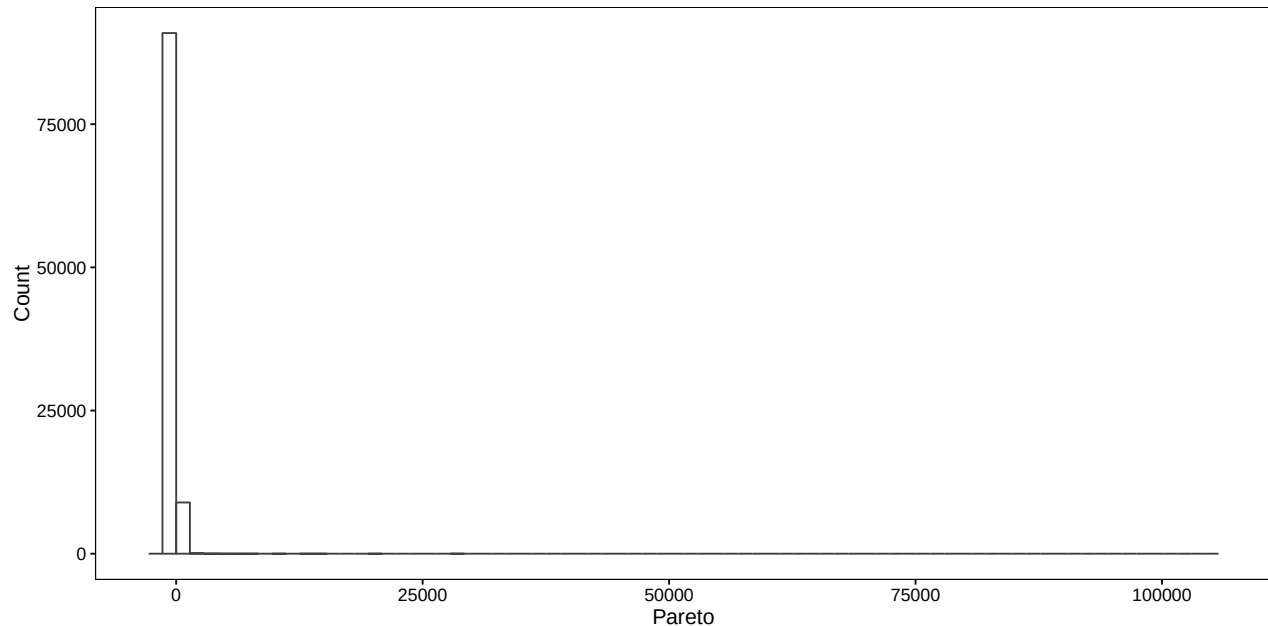
¹²Yes, that's his real name.

¹³AKA "fat tailed". But "infinite variance" is *so* much more intimidating!

¹⁴In fact, OLS is particularly bad in the case of finite variance error distributions, since it squares the errors. Quantile regression, which takes the least absolute deviation, will handle this kind of distribution better. Fun fact: one of the best things about being an applied econometrician is that you can read and use (but don't have to write) dissertations like [this one](#), from which I stole that fact.

¹⁵Before you get turned off from the Pareto distribution entirely - it does a pretty good job of describing the firm size distribution, and it makes some things analytically tractable, so it's used in a bunch of new new trade theory models. Maybe that turned you off further?

errorPlot5



```
#create Y
popY5 <- 1 + 5*popX + popError5
# put together a dataset
paretoPopData <- cbind(popY5, 1, popX) %>%
  as.tbl_df()
#name the columns
names(paretoPopData) <- c("popY", "ones", "popX")
#take a quick look
paretoPopData

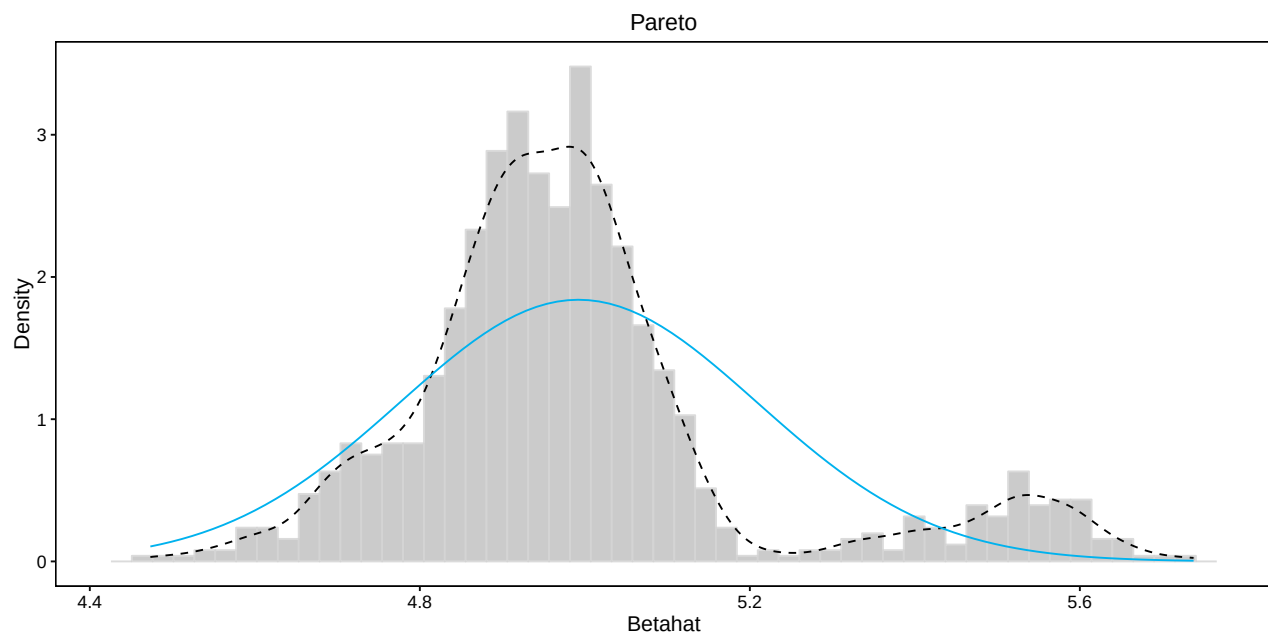
## Source: local data frame [100,000 x 3]
##
##      popY  ones  popX
##      (dbl) (dbl) (dbl)
## 1  346.77583     1  72.09039
## 2  419.36129     1  87.57732
## 3  362.72639     1  76.09823
## 4  427.91042     1  88.61246
## 5  210.67185     1  45.64810
## 6   65.35308     1  16.63718
## 7  144.00627     1  32.50954
## 8  235.44854     1  50.92243
## 9  346.83505     1  72.77053
## 10 475.87240     1  98.97369
## ..      ...     ...     ...
```

Now we can finally actually run our regressions. We'll plot the output as well (and again compare it to a Normal

distribution with the empirical mean and SD):

```
#regs  
paretoBetaHats <- genRegData(paretoPopData, "popY", c("ones", "popX"), 1000, 10000)  
  
#plot  
paretoBetaPlot <- betaHatPlot(paretoBetaHats) + ggtitle("Pareto") +  
  xlab("Betahat") + ylab("Density")
```

paretoBetaPlot



Yuh-oh. That doesn't look very Normal. Let's test it.

```
(sw4 <- (shapiro.test(paretoBetaHats$betahat))$p.value)
## [1] 1.195088e-26
```

As we suspected - not even close. It should be comforting, if slightly unsettling, to know that when the assumptions of OLS are violated, things go wacky. That said, we had to construct a kind of unusual example to get this to happen.

This concludes our tour of asymptopia. We'll pick back up next time with the first of our two weeks on non-standard standard errors.